

EXPERIMENT NO. 10

IoT-Based Smart Agriculture and Automated Irrigation Using Python in Google Colab

ABISHEK DEVARAJ (192512126)

AIM

To design and simulate an IoT-based smart agriculture system using Python in Google Colab that monitors soil moisture, temperature, and humidity and automatically controls irrigation based on soil moisture conditions.

OBJECTIVES

1. To simulate an IoT-based smart agriculture system.
2. To generate real-time soil moisture, temperature, and humidity data.
3. To monitor soil moisture continuously.
4. To automatically control the irrigation pump.
5. To detect dry and adequately moist soil conditions.
6. To visualize sensor data using graphs.
7. To calculate basic statistical values.
8. To store the collected sensor data.
9. To demonstrate automated irrigation using IoT concepts.

SOFTWARE / APPARATUS REQUIRED

- Google Colab
- Python 3.x
- pandas
- numpy
- matplotlib
- random

- time
- IPython.display

THEORY

Smart agriculture uses **IoT sensors, data analytics, and automated control systems** to improve agricultural productivity and reduce unnecessary use of water.

In a smart irrigation system, sensors continuously monitor environmental and soil conditions. A **soil-moisture sensor** determines whether sufficient water is available in the soil. Temperature and humidity sensors can also be used to understand the surrounding environmental conditions.

In this experiment, physical sensors are not connected. Instead, Python in Google Colab is used to simulate sensor readings.

The system generates:

- Soil Moisture (%)
- Temperature (°C)
- Humidity (%)

The soil moisture value is compared with a predefined threshold.

If:

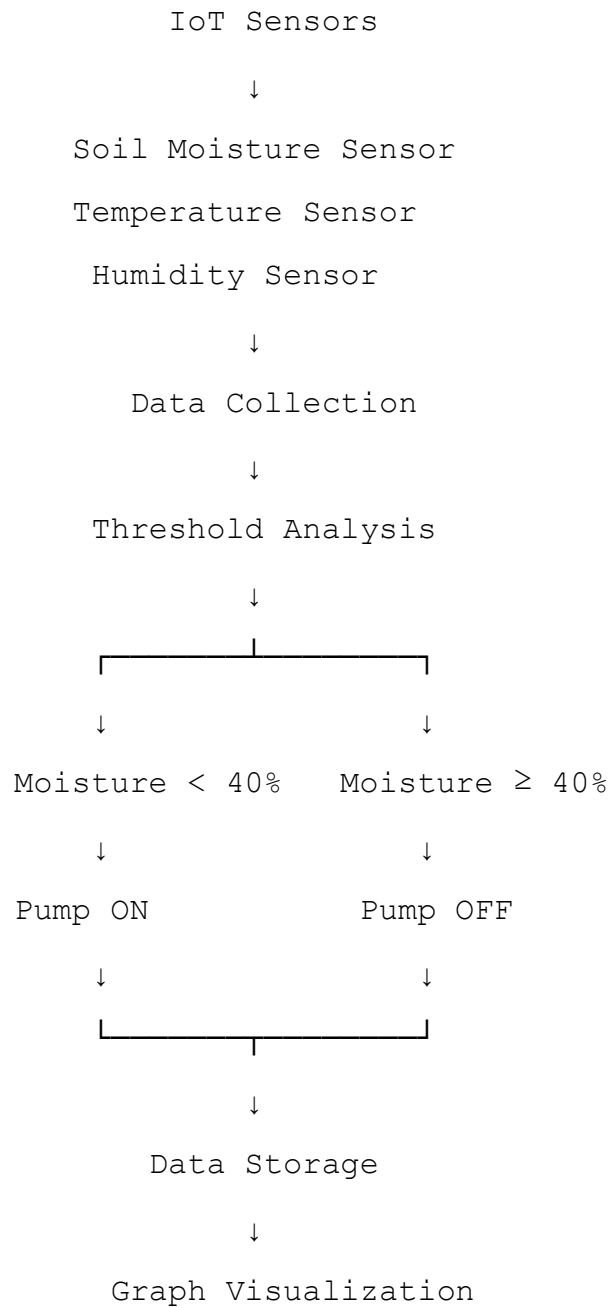
Soil Moisture < 40% → Irrigation Pump ON

If:

Soil Moisture ≥ 40% → Irrigation Pump OFF

The sensor data is displayed and visualized using graphs. The collected data is also stored in a CSV file for future analysis.

SYSTEM FLOW



THRESHOLD VALUES

Parameter	Normal/Required Condition	Action
Soil Moisture	< 40%	Pump ON
Soil Moisture	$\geq$ 40%	Pump OFF
Temperature	20–40 °C	Monitoring
Humidity	40–90%	Monitoring

ALGORITHM

1. Start the program.
2. Import the required Python libraries.
3. Initialize data-storage lists.
4. Set the soil-moisture threshold to 40%.
5. Generate soil-moisture values.
6. Generate temperature values.
7. Generate humidity values.
8. Compare soil moisture with the threshold.
9. If soil moisture is below 40%, turn the irrigation pump ON.
10. Otherwise, keep the irrigation pump OFF.
11. Store the sensor readings.
12. Display the current sensor values and pump status.
13. Plot soil moisture, temperature, and humidity graphs.
14. Repeat the process for the required number of samples.
15. Calculate the average sensor values.
16. Save the collected data as a CSV file.
17. Display the final irrigation summary.
18. Stop the program.

GOOGLE COLAB CODE

```
=====
```

```
EXPERIMENT NO. 10
```

```
IoT-Based Smart Agriculture and Automated Irrigation
```

```
Google Colab Simulation
```

```
=====
```

```
import random import time import pandas as pd import numpy as np
import matplotlib.pyplot as plt from IPython.display import
clear_output
```

```
=====
```

```
1. SYSTEM SETTINGS
```

```
=====
```

```
TOTAL_SAMPLES = 20 MOISTURE_THRESHOLD = 40
```

```
Data storage
```

```
sample_numbers = [] soil_moisture_data = [] temperature_data =
[] humidity_data = [] pump_status_data = []
```

```
=====
```

```
2. START SYSTEM
```

```
=====
```

```
print("=" * 70) print(" IoT SMART AGRICULTURE & AUTOMATED
IRRIGATION") print("=" * 70)
```

```
print() print("Soil Moisture Threshold :", MOISTURE_THRESHOLD,
"%") print("Total Samples :", TOTAL_SAMPLES) print()
print("Starting Smart Agriculture Monitoring...")

time.sleep(2)
```

```
=====
```

3. SENSOR MONITORING LOOP

```
=====
```

```
for sample in range(1, TOTAL_SAMPLES + 1):

# -----
# Generate simulated sensor values
# -----

soil_moisture = random.randint(20, 80)

temperature = round(
    random.uniform(24, 38),
    1
)

humidity = round(
    random.uniform(45, 85),
    1
)

# -----
# Automated Irrigation Decision
# -----

if soil_moisture < MOISTURE_THRESHOLD:

    pump_status = "ON"

else:
```

```

    pump_status = "OFF"

# -----
# Store data
# -----

sample_numbers.append(sample)

soil_moisture_data.append(
    soil_moisture
)

temperature_data.append(
    temperature
)

humidity_data.append(
    humidity
)

pump_status_data.append(
    pump_status
)

# -----
# Create DataFrame
# -----

df = pd.DataFrame({

    "Sample": sample_numbers,

    "Soil Moisture (%)":
    soil_moisture_data,

    "Temperature (°C)":
    temperature_data,

    "Humidity (%)":

```

```

        humidity_data,

        "Pump Status":
        pump_status_data

    })

# =====
# CLEAR PREVIOUS OUTPUT
# =====

clear_output(wait=True)

# =====
# DISPLAY CURRENT STATUS
# =====

print("=" * 70)
print("          IoT SMART AGRICULTURE & AUTOMATED IRRIGATION")
print("=" * 70)

print()

print("Current Sample      :", sample)

print(
    "Soil Moisture      :",
    soil_moisture,
    "%"
)

print(
    "Temperature        :",
    temperature,
    "°C"
)

print(
    "Humidity            :",

```



```

        humidity,
        "%"
    )

print(
    "Irrigation Pump      :",
    pump_status
)

print()

if pump_status == "ON":

    print("🔴 IRRIGATION ALERT")
    print("Soil is dry - Irrigation Pump ACTIVATED")

else:

    print("✓ Soil moisture is sufficient")
    print("Irrigation Pump DEACTIVATED")


# =====
# CURRENT SENSOR TABLE
# =====

print()
print("SENSOR MONITORING DATA")
print("-" * 70)

print(
    df.to_string(
        index=False
    )
)

# =====
# GRAPH 1 - SOIL MOISTURE
# =====

```

```
plt.figure(  
    figsize=(10, 5)  
)  
  
plt.plot(  
    sample_numbers,  
    soil_moisture_data,  
    marker="o",  
    linewidth=2,  
    label="Soil Moisture"  
)  
  
plt.axhline(  
    MOISTURE_THRESHOLD,  
    linestyle="--",  
    linewidth=2,  
    label="Irrigation Threshold (40%) "  
)  
  
plt.xlabel(  
    "Sample"  
)  
  
plt.ylabel(  
    "Soil Moisture (%) "  
)  
  
plt.title(  
    "Real-Time Soil Moisture Monitoring"  
)  
  
plt.grid(  
    True,  
    linestyle="--",  
    alpha=0.5  
)  
  
plt.legend()  
  
plt.tight_layout()
```

```
plt.show()

# =====
# GRAPH 2 - TEMPERATURE
# =====

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    sample_numbers,
    temperature_data,
    marker="s",
    linewidth=2
)

plt.xlabel(
    "Sample"
)

plt.ylabel(
    "Temperature (°C)"
)

plt.title(
    "Agricultural Field Temperature"
)

plt.grid(
    True,
    linestyle="--",
    alpha=0.5
)

plt.tight_layout()

plt.show()
```

```

# =====
# GRAPH 3 - HUMIDITY
# =====

plt.figure(
    figsize=(10, 5)
)

plt.plot(
    sample_numbers,
    humidity_data,
    marker="^",
    linewidth=2
)

plt.xlabel(
    "Sample"
)

plt.ylabel(
    "Humidity (%)"
)

plt.title(
    "Agricultural Field Humidity"
)

plt.grid(
    True,
    linestyle="--",
    alpha=0.5
)

plt.tight_layout()

plt.show()

# =====
# WAIT FOR NEXT SAMPLE

```

```

# =====

if sample != TOTAL_SAMPLES:

    time.sleep(1)

=====

4. FINAL REPORT

=====

print() print("=" * 70) print(" FINAL AGRICULTURE REPORT")
print("=" * 70)

print()

print( df.to_string( index=False ) )

=====

5. STATISTICAL ANALYSIS

=====

average_moisture = np.mean( soil_moisture_data )
average_temperature = np.mean( temperature_data )
average_humidity = np.mean( humidity_data )
pump_on_count = pump_status_data.count( "ON" )
pump_off_count = pump_status_data.count( "OFF" )

print() print("=" * 70) print(" SMART AGRICULTURE SUMMARY")
print("=" * 70)

print()

print( "Average Soil Moisture :", round(average_moisture, 2),
"%")

```

```
print( "Average Temperature :", round(average_temperature, 2),
      "°C" )

print( "Average Humidity :", round(average_humidity, 2), "%" )

print( "Pump ON Samples :", pump_on_count )

print( "Pump OFF Samples :", pump_off_count )
```

=====

6. PUMP STATUS GRAPH

=====

```
pump_values = [ 1 if status == "ON" else 0 for status in
pump_status_data ]

plt.figure( figsize=(10, 5) )

plt.step( sample_numbers, pump_values, where="mid",
linewidth=2 )

plt.yticks( [0, 1], ["OFF", "ON"] )

plt.xlabel( "Sample" )

plt.ylabel( "Irrigation Pump" )

plt.title( "Automated Irrigation Pump Status" )

plt.grid( True, linestyle="--", alpha=0.5 )

plt.tight_layout()

plt.show()
```

```
=====

7. SAVE DATA
```

```
=====

df.to_csv( "Smart_Agriculture_Irrigation_Report.csv",
index=False )

=====
```

```
8. COMPLETION MESSAGE

=====
```

```
print() print("=" * 70)

print( "IoT-Based Smart Agriculture Simulation Completed
Successfully!" )

print( "Irrigation decisions generated automatically." )

print( "Report saved as:
Smart_Agriculture_Irrigation_Report.csv" )

print("=" * 70)
```

OBSERVATION

S.No.	Parameter	Observation
1	Total Samples	20
2	Soil Moisture	Randomly generated
3	Temperature	Randomly generated
4	Humidity	Randomly generated
5	Moisture Threshold	40%
6	Irrigation Control	Automatic
7	Pump Condition	ON / OFF
8	Visualization	Line and step graphs
9	Data Storage	CSV
10	Platform	Google Colab

SAMPLE OUTPUT

```
=====
=====

      IoT SMART AGRICULTURE & AUTOMATED IRRIGATION

=====
=====
```

Current Sample : 8

Soil Moisture : 31 %

Temperature : 32.6 °C

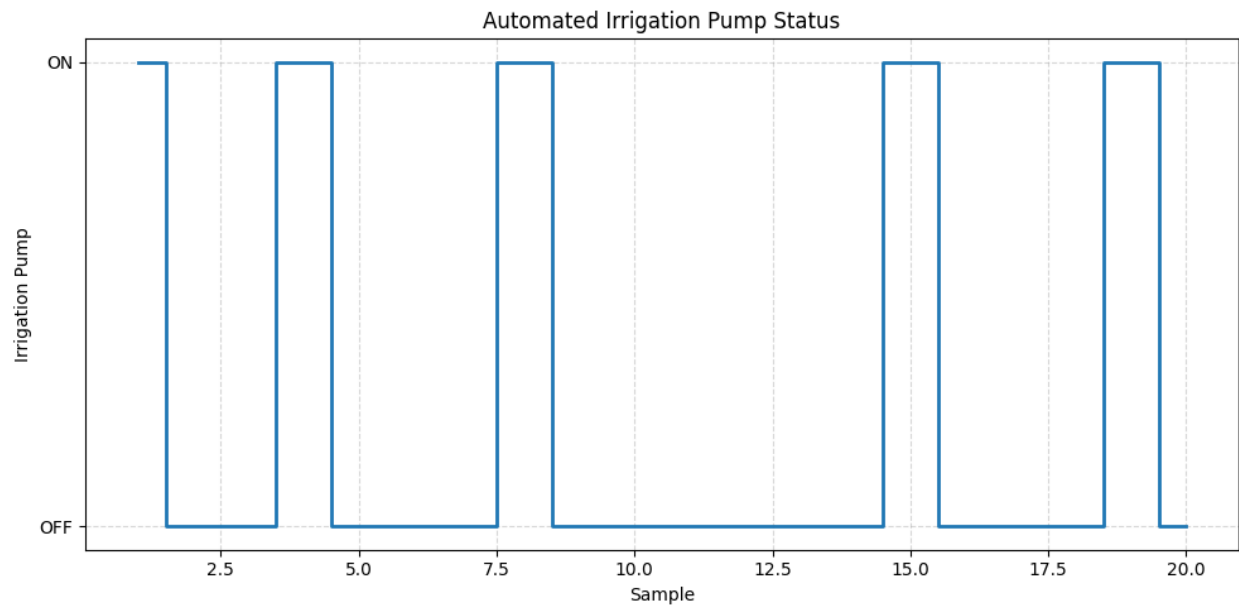
Humidity : 67.4 %

Irrigation Pump : ON

 IRRIGATION ALERT

Soil is dry - Irrigation Pump ACTIVATED

OUTPUT



RESULT

The **IoT-Based Smart Agriculture and Automated Irrigation System** was successfully simulated using Python in Google Colab. The system generated soil-moisture, temperature, and humidity values and automatically controlled the simulated irrigation pump based on the soil-moisture threshold.